

ODMA–URA research record

# From explicit sparse codebooks to generated supports

Why generation and inversion are both needed; tests 028–029

|         |                                              |
|---------|----------------------------------------------|
| Stage   | 5 of 5                                       |
| Updated | 5 September 2026                             |
| Status  | Implementation verified; HPC results pending |

This report is a chronological research record. Measured results, interpretations, and open hypotheses are labelled separately. Detailed numerical ledgers remain in **results/**.

## 1 The result we are trying to keep

A  $B$ -bit unsourced message has one of  $M = 2^B$  values. In the explicit model, message  $m$  is column  $\phi_m$  of

$$\Phi = [\phi_0, \dots, \phi_{M-1}] \in \mathbb{R}^{n \times M}, \quad y = \Phi a + z = \sum_{m=0}^{M-1} a_m \phi_m + z,$$

where  $a_m$  is the number of active devices sending message  $m$ . Every column satisfies  $\|\phi_m\|_2 = 1$ .

Job 027 showed that these columns need not be dense. A random column with only  $s$  nonzeros remained close to the dense reference over a broad range of  $s$ , while ODMA was much worse at equal density. The useful object therefore appears to be a diverse sparse global codebook, rather than sparsity alone.

The difficulty is scale. Even a sparse explicit codebook stores  $s2^B$  values and its decoder maintains a score for all  $2^B$  messages. This is practical at  $B = 12$  or  $14$  and impossible at  $B \simeq 100$ . We want to retain the same kind of sparse codeword without storing one independently for every message.

## 2 What a generated codebook must provide

Write the message as  $w \in \{0, 1\}^B$ . A procedural encoder replaces stored columns by two functions:

$$S(w) = \{\rho_0(w), \dots, \rho_{s-1}(w)\}, \quad \phi(w) = \sum_{t=0}^{s-1} \alpha_t(w) e_{\rho_t(w)}, \quad \sum_t |\alpha_t(w)|^2 = 1.$$

The first function generates the occupied channel uses; the second generates their real or complex amplitudes. The binary values are message bits, not channel symbols. If both functions use compact state, a transmitter can create only the few codewords it sends, with no  $2^B$  object.

This solves only the forward problem. The receiver sees a noisy sum of several generated codewords and must recover the messages. A rule that is cheap to evaluate can still require trying all  $2^B$  inputs to invert. A scalable design must therefore ultimately supply both

codeword( $w$ )    and    propose( $y$ )  $\longrightarrow$  a short list containing the transmitted messages.

“Invertible” has two levels here. Distinct messages must first produce distinct generated descriptions. More importantly, likely messages must be recoverable from a noisy multiuser sum without exhaustive search. The present work tests the first requirement and whether its structure harms recovery at small  $B$ ; it does not yet solve the second.

## 3 A simple generated support

Set  $T = s$  and divide the  $n$  channel uses into  $T$  disjoint groups (tables), each containing  $R = n/T$  rows. One hash per table selects one row:

$$\rho_t(w) = tR + h_t(w), \quad h_t(w) \in \{0, \dots, R-1\}.$$

Every codeword then has exactly one nonzero in each table. This changes the model class: there are  $R^T$  allowed supports rather than all  $\binom{n}{s}$  supports. It is acceptable to test, rather than assume, this concession because:

- there is no label shortage when  $B \leq T \log_2 R$ ;

- uniform table choices give expected pair overlap  $T/R = s^2/n$ , the same first-order value as uniform size- $s$  supports;
- a balanced random-table control can measure the cost of this restriction directly against arbitrary sparse support at the same  $s$ , amplitudes, decoder, and operating point.

Thus a null performance gap would be a useful simplification, not a proof that the restriction is harmless everywhere.

For a compact rule, take  $R = 2^r$  and use binary affine hashes

$$h_t(w) = A_t w \oplus b_t, \quad A_t \in \mathbb{F}_2^{r \times B}, \quad b_t \in \mathbb{F}_2^r.$$

Here  $\mathbb{F}_2$  is simply binary arithmetic: addition is XOR. Full row rank of each  $A_t$  balances its table. Full rank of the stacked matrix  $[A_0^T \cdots A_{T-1}^T]^T$  makes the complete hash tuple unique for every message. This is exact isolated-message invertibility; it is not yet a multiuser decoder.

Linear hashes add another testable restriction. If two messages differ by  $d$ , they collide in table  $t$  exactly when  $A_t d = 0$ . Job 028 therefore compares a random valid hash with one selected from 128 candidates to reduce the worst and tail collision counts. The discrete matrices  $A_t, b_t$  are designed offline, not learned by gradient descent, so rank, balance, and uniqueness remain exact.

## 4 The small- $B$ matrix used by the current decoder

At  $B = 14$  we may materialise the generated rule without changing the existing decoder. Let

$$(P_t)_{jm} = \mathbf{1}\{j = h_t(w_m)\}, \quad D_t = \text{diag}(\alpha_{t0}, \dots, \alpha_{t,M-1}).$$

Then

$$\Phi = \begin{bmatrix} P_0 D_0 \\ P_1 D_1 \\ \vdots \\ P_{T-1} D_{T-1} \end{bmatrix} \in \mathbb{R}^{TR \times 2^B}, \quad \Phi_{tR+j,m} = \alpha_{tm} \mathbf{1}\{j = h_t(w_m)\}.$$

This equation is the bridge from the generator to the explicit experiment. It proves exactly which entries exist and lets D0/D1 use their unchanged operations  $y - \Phi \hat{a}$  and  $\Phi^T(y - \Phi \hat{a})$ . Because table rows are disjoint,  $\sum_t |\alpha_{tm}|^2 = 1$  gives unit codeword energy exactly.

In job 028 the amplitudes are fixed Gaussian draws. In job 029 they are learned jointly with the selected decoder while the support remains fixed. If  $S \in \{0, 1\}^{n \times M}$  is the support mask and  $V$  contains trainable values, then

$$\Phi = S \odot V, \quad \frac{\partial \mathcal{L}}{\partial V} = S \odot \frac{\partial \mathcal{L}}{\partial \Phi}.$$

The mask prevents a zero from becoming nonzero. After every optimizer step, each column is projected back to unit norm. Dense learning uses the same rule with  $S$  equal to all ones. This makes the comparison fair and guarantees the transmit-power constraint after training.

These learned amplitudes are still stored per message. Job 029 asks whether a fixed generated *support* leaves enough freedom for useful encoder-decoder co-adaptation; it is not yet a fully procedural large- $B$  encoder. A compact amplitude rule and a sub-exhaustive proposer are later requirements.

## 5 What the two experiments test

Job 028 contains four sparse families at  $B = 14, n = 256, T \in \{16, 32\}$ , D0/D1, and two seeds, with dense context. Job 029 deliberately uses fewer families: learned dense, iid sparse, and selected hash at the same two supports. It trains each encoder and its chosen decoder together for 120 epochs, records the full loss curve, and measures geometry before and after learning. D0 and D1 receive the same number of updates; the earlier budgets were 80 and 20 epochs.

The comparisons are deliberately sequential:

1. balanced tables versus iid sparse tests the cost of predetermined groups;
2. random linear hashes versus balanced tables tests the added cost of affine/XOR structure;
3. selected versus random hashes tests whether collision-aware offline design helps;
4. joint versus fixed amplitudes tests whether co-adaptation removes an apparent sparse tail or training floor;
5. selected hash versus iid sparse after joint learning is the final support-generator comparison.

The local test also builds  $A \in \mathbb{F}_2^{64 \times 2 \times 100}$  for  $B = 100, n = 256, T = 64, R = 4$ , checks its stacked rank, and generates the rows of a few supplied messages. This is a cheap regression test that the support rule itself has no hidden  $2^B$  axis. It neither constructs  $\Phi$  nor runs D0/D1, so it is not evidence of scalable decoding and is not a performance experiment.

## 6 Decision after the jobs return

The first decision remains causal: does balanced table support match iid sparse in job 028? If not, the proposed skeleton is rejected. If it does, the linear-hash comparisons identify whether the compact affine rule also survives. Job 029 then shows whether conclusions based on fixed amplitudes persist when the permitted continuous parameters and decoder are co-adapted.

Only a generated skeleton that remains competitive with iid sparse should advance to the real large- $B$  problem: procedural amplitudes plus a high-recall candidate proposer whose memory and runtime do not scale as  $2^B$ .

**Evidence status:** The algebra, fixed-support gradient mask, unit-energy projection, and local joint-learning paths are verified. Jobs 028 and 029 have no HPC performance results yet.